

ЛАБОРАТОРНА РОБОТА №7

КЛАСИ. Ч. 1

Мета роботи: ознайомитися з ООП в мові Python

Хід роботи:

Завдання 1. Реалізувати клас Person, який відображає запис в книзі контактів.

Клас має 4 атрибута: - surname - рядок - прізвище контакту (обов'язковий) - first_name - рядок - ім'я контакту (обов'язковий) - nickname - рядок - псевдонім (опціональний) - birth_date - об'єкт datetime.date (обов'язковий) Кожен виклик класу повинен створювати екземпляр (інстанс) класу із зазначеними атрибутами.

Також клас має 2 методи: - get_age()

- рахує вік особи в повних роках на дату виклику і повертає рядок виду: "25"; - get_fullname()

- повертає рядок, що відображає повне ім'я (прізвище + ім'я) контакту; Примітка: при створенні атрибута birth_date з рядка типу "2002-12-31" необхідно:

- визначити яка інформація потрібна для створення об'єкта datetime.date, - отримати ці дані з рядка

- розібрати її (дістати з неї окремо, рік, місяць, число),

- на підставі цієї інформації створити спеціальний об'єкт datetime.date,

- помістити цей спец.об'єкт в атрибут екземпляра класу

					ДУ «Житомирська політехніка».25.F2.02.000 – Лр7			
Змн.	Арк.	№ докум.	Підпис	Дата				
Розроб.		Гришкова Е. А.			Звіт з лабораторної роботи		Літ.	Арк.
Перевір.		Желізко В. В.						1
Керівник								5
Н. контр.							ФІКТ Гр. ВТк-25-1	
Зав. каф.								

Лістинг коду:

```
from datetime import date

class Person:
    def __init__(self, surname, first_name, birth_date, nickname=None):
        self.surname = surname
        self.first_name = first_name
        self.nickname = nickname

        year, month, day = map(int, birth_date.split('-'))
        self.birth_date = date(year, month, day)

    def get_age(self):
        today = date.today()
        age = today.year - self.birth_date.year - (
            (today.month, today.day) < (self.birth_date.month, self.birth_date.day)
        )
        return str(age)

    def get_fullname(self):
        return f"{self.surname} {self.first_name}"

if __name__ == "__main__":
    person = Person("Шевченко", "Тарас", "1995-03-09", "Kobzar")

    print(f"Повне ім'я: {person.get_fullname()}")
    print(f"Вік: {person.get_age()}")
    print(f"Нікнейм: {person.nickname}")
```

Результат виконання:

```
Повне ім'я: Шевченко Тарас
Вік: 30
Нікнейм: Kobzar
```

розроблено клас Person, що моделює об'єкт запису в книзі контактів. Програмна реалізація базується на принципах об'єктно-орієнтованого програмування. Конструктор класу __init__ забезпечує ініціалізацію обов'язкових атрибутів (прізвище, ім'я, дата народження) та опціонального параметра псевдоніма. Окрему увагу приділено обробці дати: вхідний рядок формату "РРРР-ММ-ДД" парситься за допомогою функції map, після чого створюється об'єкт типу datetime.date. Метод get_age реалізує алгоритм обчислення повної кількості років шляхом порівняння поточної дати з датою народження, враховуючи, чи

		Гришкова Е.А.			ДУ «Житомирська політехніка». 25.F2.02.000 – Лр7	Арк.
		Желіско В.В.				2
Змн.	Арк.	№ докум.	Підпис	Дата		

відбувся день народження у поточному календарному році. Метод `get_fullname` виконує конкатенацію рядкових значень прізвища та імені для формування повного ідентифікатора особи.

Завдання 2. Написати функцію `modifier(filename)`, яка приймає ім'я файлу і повинна: - прочитати дані з переданого файлу; - створити об'єкти класу `Person` на підставі отриманих даних; - модифікувати дані в файлі: а) додати графу повного імені (`fullname`) після графи з ім'ям (`name`) б) додати графу з віком (`age`) в кінець. На виході отримати файл, розширений зазначеним чином.

Лістинг коду:

```
import csv
from datetime import date

class Person:
    def __init__(self, surname, first_name, birth_date, nickname=None):
        self.surname = surname
        self.first_name = first_name
        self.nickname = nickname
        year, month, day = map(int, birth_date.split('-'))
        self.birth_date = date(year, month, day)

    def get_age(self):
        today = date.today()
        age = today.year - self.birth_date.year - (
            (today.month, today.day) < (self.birth_date.month, self.birth_date.day)
        )
        return str(age)

    def get_fullname(self):
        return f"{self.surname} {self.first_name}"

def modifier(filename):
    contacts = []
    with open(filename, 'r', encoding='utf-8') as file:
        reader = csv.DictReader(file)
        fieldnames = list(reader.fieldnames)
        for row in reader:
            p = Person(row['surname'], row['first_name'], row['birth_date'],
row.get('nickname'))
            row['fullname'] = p.get_fullname()
            row['age'] = p.get_age()
            contacts.append(row)
```

```

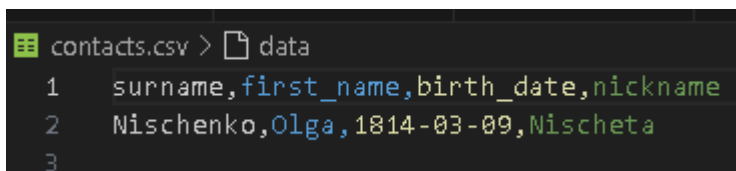
new_fieldnames = list(fieldnames)
new_fieldnames.insert(new_fieldnames.index('first_name') + 1, 'fullname')
new_fieldnames.append('age')

with open(filename, 'w', encoding='utf-8', newline='') as file:
    writer = csv.DictWriter(file, fieldnames=new_fieldnames)
    writer.writeheader()
    writer.writerows(contacts)

if __name__ == "__main__":
    test_file = "contacts.csv"
    with open(test_file, 'w', encoding='utf-8', newline='') as f:
        f.write("surname,first_name,birth_date,nickname\n")
        f.write("Nischenko,Olga,1814-03-09,Nischeta\n")

```

Результат виконання:



```

contacts.csv > data
1  surname,first_name,birth_date,nickname
2  Nischenko,Olga,1814-03-09,Nischeta
3

```

реалізовано функцію `modifier`, призначену для автоматизації обробки структурованих даних у файловій системі. Процес обробки складається з трьох етапів: зчитування, трансформації та запису. Використання модуля `csv.DictReader` дозволяє десеріалізувати дані з файлу безпосередньо у словники, що спрощує доступ до полів за ключами.

Для кожного рядка вхідних даних створюється екземпляр класу `Person`, за допомогою якого генеруються нові ознаки: повне ім'я та вік. Алгоритм модифікації структури файлу передбачає динамічне оновлення списку заголовків (`fieldnames`), де поле `fullname` вставляється після імені, а `age` додається в кінець списку. Фінальний етап роботи функції забезпечує перезапис файлу з оновленою структурою стовпців та збереженням цілісності модифікованих даних.

		Гришкова Е.А.			ДУ «Житомирська політехніка». 25.F2.02.000 – Лр7	Арк.
		Желізко В.В.				4
Змн.	Арк.	№ докум.	Підпис	Дата		

Посилання на репозиторій: <https://github.com/wghtl4s/Python>

Висновки: У ході лабораторної роботи було опановано базові принципи об'єктно-орієнтованого програмування на мові Python. На прикладі класу Person реалізовано механізм створення об'єктів із заданими атрибутами та методами для обробки персональних даних.

		Гришкова Е.А.			ДУ «Житомирська політехніка». 25.F2.02.000 – Лр7	Арк.
		Желізко В.В.				5
Змн.	Арк.	№ докум.	Підпис	Дата		